

Long Path Problem

Gegeben: ein Graph G und eine Länge B

Gesucht: gibt es einen Pfad der Länge B ?

hat also B Kanten und
 $B+1$ verschiedene Knoten

○—○—○—○—○

$B=4$ Kanten und

$B+1=5$ Knoten

Schwierigkeit: Falls das Hamiltonkreisproblem NP-schwer ist, dann ist auch Long-Path NP-schwer

Beweis: mittels Kontraposition ($A \rightarrow B \equiv \neg B \rightarrow \neg A$)

Annahme: Long-Path ist in polynomieller Zeit lösbar

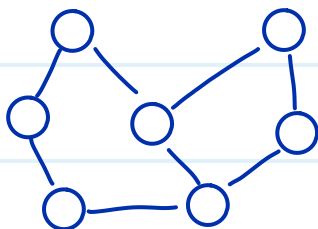
Zu zeigen: Hamiltonkreisproblem ist in polynomieller Zeit lösbar

Sei G ein beliebiger Graph mit n Knoten

Wir konstruieren G' wie folgt:

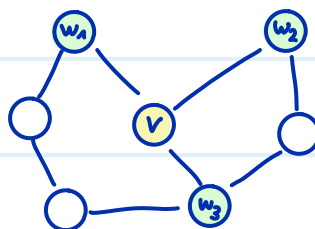
- Sei G' eine Kopie von G , ausser dass wir einen beliebigen Knoten v entfernen
- Dann erstelle für alle Kanten $\{v, w_i\}$ aus G einen neuen Knoten $\hat{w}_i$
und eine neue Kante $\{w_i, \hat{w}_i\}$ in G'
- G' hat also $\underbrace{(n-1)}_{\text{alle ausser } v} + \underbrace{\deg(v)}_{\substack{\text{für jede Kante von } v \\ \text{gibt es einen neuen Knoten } \hat{w}_i \\ \text{und } \deg(v) \leq n-1}} \leq 2n-2$ viele Knoten

G :



(G hat Hamiltonkreis)

wähle v

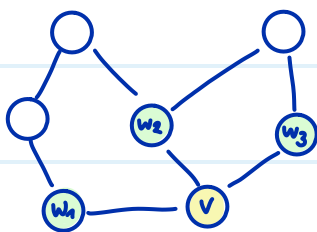


G' :

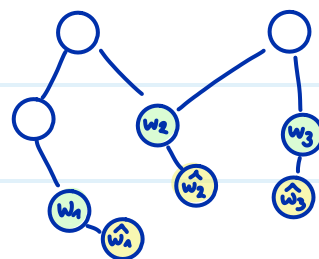


(G' hat Pfad der Länge n)

wähle v (andere Möglichkeit)



G' :



(G' hat Pfad der Länge n)

Zu zeigen: G hat einen Hamiltonkreis $\Leftrightarrow G'$ hat einen Pfad der Länge n

" $\Rightarrow$ " Annahme: $(v_1, v_2, \dots, v_n, v_1)$ ein Hamiltonkreis in G

Sei $v = v_1$ w.l.o.g. (wir können den Kreis rotieren sodass $v = v_1$)

$\Rightarrow (\hat{v}_2, v_2, v_3, \dots, v_n, \hat{v}_n)$ ist ein Pfad der Länge n in G'

$\underbrace{\hat{v}_2, v_2}_{v_2 \text{ war Nachbar von } v, \text{ deshalb gibt es } \hat{v}_2}$

$\underbrace{v_n, \hat{v}_n}_{v_n \text{ war Nachbar von } v \text{ und, deshalb gibt es } \hat{v}_n}$

" $\Leftarrow$ " Annahme: $(u_0, u_1, \dots, u_n)$ ist ein Pfad der Länge n in G'

$\Rightarrow$ die Knoten $u_1, \dots, u_{n-1}$ haben alle Grad ≥ 2 (weil sie innere Knoten im Pfad sind)

$\Rightarrow$ die Knoten $u_1, \dots, u_{n-1}$ sind genau die $n-1$ überlebenden Knoten aus G

(weil alle anderen Knoten $\hat{w}_i$ Grad 1 haben)

$\Rightarrow u_0 = \hat{w}_i$ und $u_n = \hat{w}_j$ sind zwei der neuen Knoten in G' (einzige übrige Möglichkeit)

$\Rightarrow u_1$ und u_{n-1} haben eine Kante zu v in G (weil sie eine Kante zu $\hat{w}_i / \hat{w}_j$ in G' haben)

$\Rightarrow (v, u_1, u_2, \dots, u_{n-1}, v)$ ist ein Hamiltonkreis in G

Schlussfolgerung: Falls Long-Path in polynomieller Zeit $O(t(n))$ entscheidbar ist, (t ist irgendeine funktion)

dann ist auch das Hamiltonkreisproblem in polynomieller Zeit $O(t(2n-2) + n^2)$

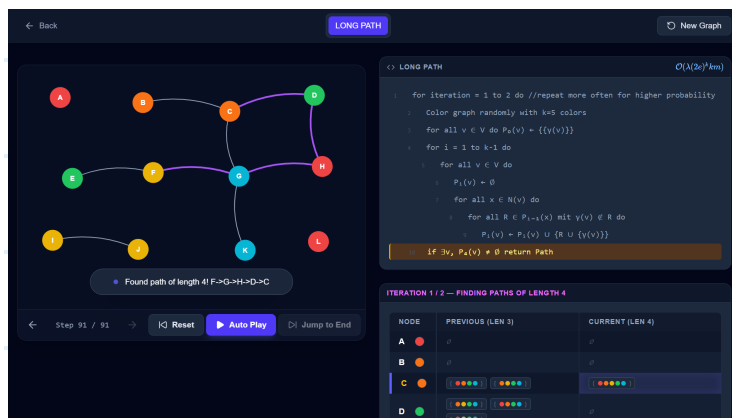
entscheidbar.

Long-Path
auf G'

Konstruktion
von G'

Monte Carlo Long Path

Gesucht: gibt es einen Pfad der Länge $k-1$? (also bestehend aus k Knoten)



1.) Färbe die Knoten mit einer zufällig gleichverteilten Färbung $\gamma: V \rightarrow [k]$

$\underbrace{\quad}_{k \text{ viele Farben}}$

• ein Pfad heisst **bunt** wenn alle Farben seiner Knoten paarweise verschieden sind

• Falls ein Pfad P der Länge $k-1$ existiert, dann existiert mit Wahrscheinlichkeit $\geq e^{-k}$

auch ein bunter Pfad der Länge $k-1$

Beweis: $\Pr[\exists \text{ bunter Pfad der Länge } k-1] \geq \Pr[\text{Pfad } P \text{ ist bunt}]$

$$= \frac{k^1}{k^k} \rightarrow \begin{array}{l} \text{Anzahl bunte Färbungen} \rightarrow \text{ziehen ohne} \\ \text{zurücklegen} \\ \text{Anzahl aller Färbungen} \rightarrow \text{ziehen mit} \\ \text{zurücklegen} \end{array}$$

$$\underline{\underline{\geq e^{-k}}}$$

$$\text{weil } e^k = \sum_{n=0}^{\infty} \frac{k^n}{n!} = 1 + k + \frac{k^2}{2!} + \frac{k^3}{3!} + \dots$$

$$\Rightarrow e^k \geq \frac{k^k}{k^1} \quad \text{alles weglassen ausser } k\text{-ten Term}$$

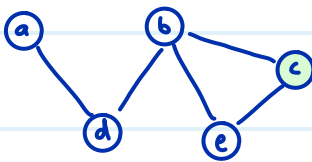
$$\Rightarrow e^{-k} \leq \frac{k^1}{k^k} \quad \text{beide Seiten hoch } -1, \text{ Vergleichszeichen dreht sich um}$$

2.) DP Algorithmus bestimmt (mit absoluter Sicherheit) ob es einen Pfad der Länge $k-1$ gibt

Teilproblem: $P_i(v) = \left\{ S \subseteq [k] \mid \begin{array}{l} |S| = i+1 \text{ und } \exists \text{ ein in } v \text{ endender, genau mit den} \\ \text{Farben aus } S \text{ gefärbter bunter Pfad der Länge } i \end{array} \right\}$

S ist eine Menge von $i+1$ Farben

Bsp



Farblegende: 1 = gelb, 2 = grün, 3 = rot, 4 = violett

Bestimme $P_1(c) = \{ \{2, 3\}, \{2, 1\} \}$

$P_2(a) = \{ \{4, 2, 3\} \}$

$P_2(b) = \{ \{3, 2, 4\}, \{3, 2, 1\} \}$

$P_0(e) = \{ \{1\} \}$

Base Case: $P_0(v) = \{ \{g(v)\} \}$ für alle Knoten $v \in V$

Rekursion: $P_i(v) = \bigcup_{x \in N(v)} \left\{ R \cup \{g(v)\} \mid \begin{array}{l} R \in P_{i-1}(x) \text{ und } g(v) \notin R \end{array} \right\}$

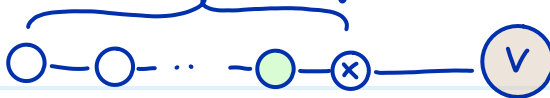
$x \in N(v)$: x muss ein Nachbar von v sein, damit wir den Pfad verlängern können

Farbset des verlängerten Pfades

$\exists$ ein bunter in x endender Pfad mit genau den Farben aus R der Länge $i-1$

die Farbe von v wurde vorher noch nicht verwendet

muss ein bunter Pfad der Länge $i-1$ sein



$R = \{ \dots, \text{green dot}, \dots \}$ und $\text{brown dot} \notin R$

$R \cup \{g(v)\} = \{ \dots, \text{green dot}, \text{brown dot} \}$

ITERATION 2 / 2 — FINDING PATHS OF LENGTH 3

NODE	PREVIOUS (LEN 2)	CURRENT (LEN 3)
A	[green, red]	[green, red, blue]
B	[green, red]	
C	[green, red]	
D	[green, red]	[green, red, blue]
E	[green, red]	[green, red, blue]
F	[green, red]	[green, red, blue]
G	[green, red]	[green, red, blue]
H	[green, red]	[green, red, blue]
I	[green, red]	[green, red, blue]
J	[green, red]	[green, red, blue]
K	[green, red]	[green, red, blue]
L	[green, red]	[green, red, blue]

Lösung: $\exists$ bunter Pfad der Länge $k-1 \Leftrightarrow \bigcup_{v \in V} P_{k-1}(v) \neq \emptyset \quad (\Leftrightarrow \exists v \in V: P_{k-1}(v) \neq \emptyset)$

Laufzeit: Base case in $O(n)$, danach wird $\text{Bunt}(G, i)$ $k-1$ mal ausgeführt

BUNT(G, i) macht den nächsten Rekursionsschritt

1: for all $v \in V$ do

2: $P_i(v) \leftarrow \emptyset$

3: for all $x \in N(v)$ do es gibt genau $\deg(v)$ viele Nachbarn

4: for all $R \in P_{i-1}(x)$ mit $\gamma(v) \notin R$ do

5: $P_i(v) \leftarrow P_i(v) \cup \{R \cup \{\gamma(v)\}\}$

$|R| = i$ viele Elemente überprüfen und kopieren

$$O(\text{Bunt}(G, i)) = O\left(\sum_{v \in V} \deg(v) \cdot |P_{i-1}(v)| \cdot i\right)$$

$$\begin{aligned} &\hookrightarrow |P_{i-1}(v)| \leq \binom{k}{i} \\ &\text{weil } P_{i-1}(v) \subseteq \binom{[k]}{i} \end{aligned}$$

$$\leq O\left(\sum_{v \in V} \deg(v) \cdot \binom{k}{i} \cdot i\right)$$

$$\leq O\left(2m \cdot \binom{k}{i} \cdot i\right)$$

$$\hookrightarrow \text{Handschlag-Lemma } \sum_{v \in V} \deg(v) = 2m$$

$\hookrightarrow 2$ weglassen

$$\leq O\left(\binom{k}{i} \cdot m \cdot i\right)$$

$$\text{Total: } O\left(\sum_{i=1}^{k-1} \binom{k}{i} m \cdot i\right)$$

$$\hookrightarrow i \leq k$$

$$\leq O\left(\sum_{i=1}^{k-1} \binom{k}{i} m k\right)$$

$\hookrightarrow$ Summe von 0 bis k statt 1 bis $k-1$

$$\leq O\left(\sum_{i=0}^k \binom{k}{i} m k\right)$$

$$\hookrightarrow \sum_{i=0}^k \binom{k}{i} = 2^k$$

folgt aus dem Binomialsatz, der aber für diesen Kurs nicht weiter relevant ist

$$= \underline{\underline{O(2^k m k)}}$$

Bisher: Monte-Carlo Algorithmus mit einseitigem Fehler

- Immer korrekt für NEIN-Instanzen:

$$\Pr[\text{kein Pfad der Länge } k-1 \text{ wird gefunden} \mid \nexists \text{ Pfad der Länge } k-1] = 1$$

- Mit Wahrscheinlichkeit $\geq e^{-k}$ korrekt für JA-Instanzen:

$$\Pr[\text{Pfad der Länge } k-1 \text{ wird gefunden} \mid \exists \text{ Pfad der Länge } k-1] \geq e^{-k}$$

3.) Fehlerreduktion: Sei $\delta = e^{-\lambda}$ unser Zielfehler.

$\varepsilon = e^{-k}$ ist die aktuelle Korrektheit

$$\text{Anzahl Wiederholungen: } N = \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil = \lceil e^k \ln(e^\lambda) \rceil = \underline{\underline{\lceil \lambda e^k \rceil}}$$

$\xrightarrow{\quad \quad \quad}$
 $\varepsilon \text{ und } \delta \text{ einsetzen} \quad \quad \ln(e^\lambda) = \lambda$

$$\text{Laufzeit insgesamt } O(\underbrace{2^k}_{\text{DP}} \cdot \underbrace{\lambda e^k}_{\text{Wiederholungen}}) = \underline{\underline{O(\lambda (2e)^k km)}}$$

Für $k \leq O(\log n)$ ist die Laufzeit polynomiell:

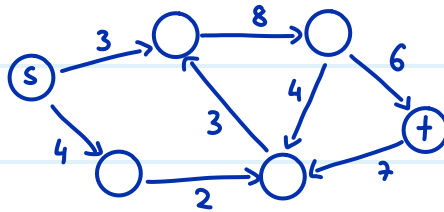
$$O(\lambda (2e)^{\log(n)} \log(n) m) = O(\lambda n^{\log(2e)} \log(n) m)$$

weil $a^{\log b} = b^{\log a}$
 $\log(2e)$ ist konstant

Flows

Definition Netzwerk: $N = (V, A, c, s, t)$ wobei

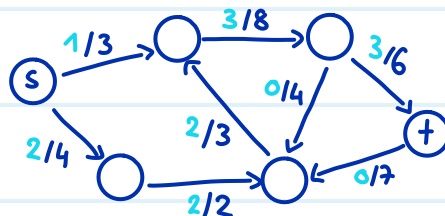
- $G = (V, A)$ ist ein gerichteter Graph (A für Arcs)
- $s \in V$ ist die source/Quelle
- $t \in V$ ist das target/Senke
- $c: A \rightarrow \mathbb{R}_0^+$ ist die Kapazitätsfunktion, die jeder Kante $e \in A$ eine Kapazität $c(e)$ zuweist



Definition Fluss: eine Funktion $f: A \rightarrow \mathbb{R}_0^+$, die jeder Kante $e \in A$ einen Flusswert $f(e)$ zuweist mit folgenden Bedingungen:

Zulässigkeit: $\forall e \in A: 0 \leq f(e) \leq c(e)$

Flusserhaltung: $\forall v \in V \setminus \{s, t\}: \underbrace{\sum_{\substack{u \in V \\ (u,v) \in A}} f(u,v)}_{\text{alles was rein fließt}} = \underbrace{\sum_{\substack{u \in V \\ (v,u) \in A}} f(v,u)}_{\text{alles was raus fließt}}$



hat Flusswert = 3

ganzzahliger Fluss: falls $\forall e \in A: f(e) \in \mathbb{Z}$

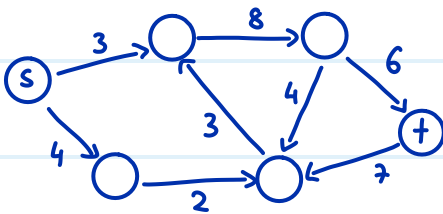
Wert des Flusses: $\text{val}(f) = \text{netoutflow}(s) = \sum_{\substack{u \in V \\ (s,u) \in A}} f(s,u) - \sum_{\substack{u \in V \\ (u,s) \in A}} f(u,s)$

was aus s raus fließt
was in s hinein fließt

Lemma Skript
 $\text{netinflow}(t) = \sum_{\substack{u \in V \\ (u,t) \in A}} f(u,t) - \sum_{\substack{u \in V \\ (t,u) \in A}} f(t,u)$

was in t hinein fließt
was aus t raus fließt

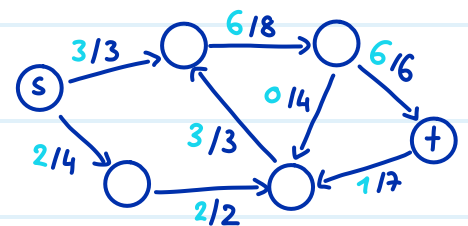
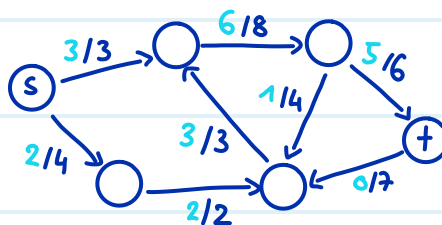
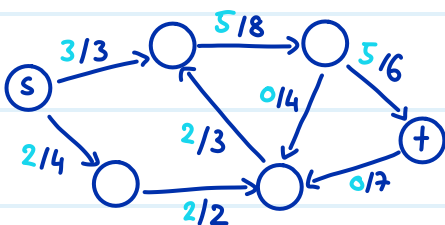
Aufgabe: bestimme den maximalen Flusswert für folgendes Netzwerk:



MaxFlow = 5

Finde drei verschiedene Flüsse in diesem Netzwerk, die alle den maximalen

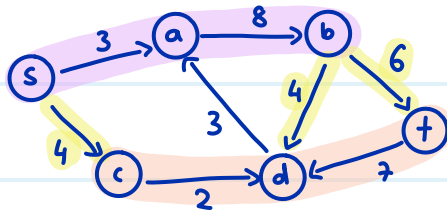
Wert haben



s-t-Schnitt: eine Partition (S, T) von V sodass $s \in S$ und $t \in T$
 das heisst $S \cup T = V$
 und $S \cap T = \emptyset$

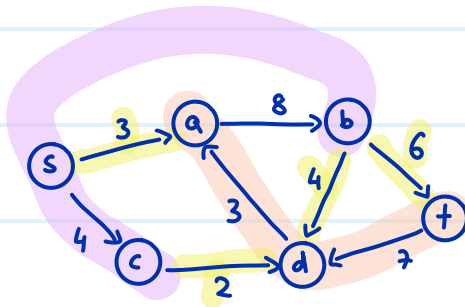
Kapazität eines s-t-Schnitts: $cap(S, T) = \sum_{(u,v) \in (S \times T) \cap A} c(u,v)$
 alle Kanten die von S zu T gehen

Bsp



Sei $S = \{s, a, b\}$ und $T = \{t, c, d\}$

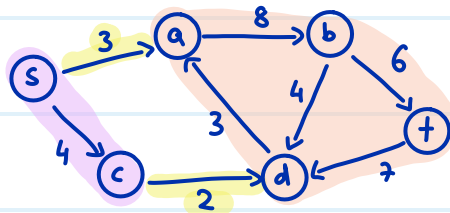
dann ist $cap(S, T) = 4 + 3 + 6 = \underline{\underline{14}}$



Sei $S = \{s, b, c\}$ und $T = \{t, a, d\}$

Berechne $cap(S, T) = 3 + 2 + 4 + 6 = \underline{\underline{15}}$

Finde einen s-t-Schnitt mit $cap(S, T) = 5$



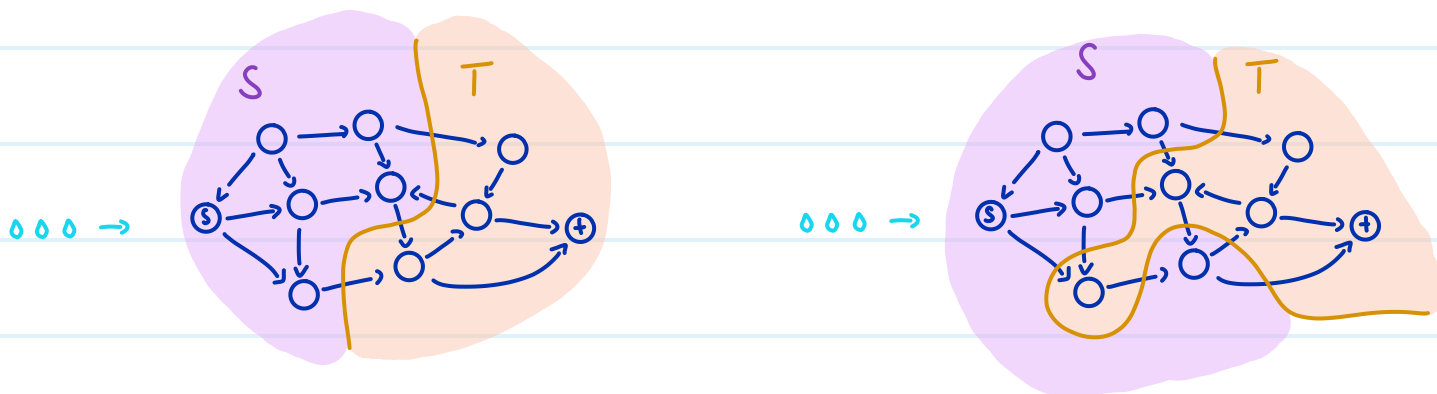
Sei $S = \{s, c\}$ und $T = \{t, a, b, d\}$

dann ist $cap(S, T) = 3 + 2 = \underline{\underline{5}}$

Lemma 3.8. Für einen beliebigen Fluss und einen beliebigen s-t-Schnitt (S, T) gilt:

$$val(f) \leq cap(S, T)$$

Intuition: Jeder einzelne "Tropfen" muss irgendwann die Grenzlinie von S nach T überqueren



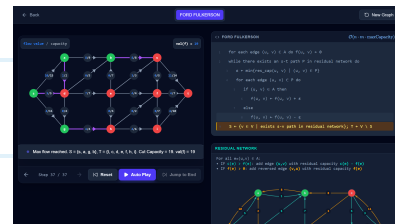
Maxflow-Mincut Theorem: $\max_{\text{Fluss } f} \text{val}(f) = \min_{(S,T) \text{ s-t-Schnitt}} \text{cap}(S,T)$

$\Rightarrow$ um zu beweisen, dass ein Fluss f maximal ist, reicht es, einen s-t-Schnitt zu finden sodass $\text{val}(f) = \text{cap}(S,T)$

Beweis mittels Ford-Fulkerson Algorithmus:

FORD-FULKERSON(V, A, c, s, t)

- | | |
|--|---|
| 1: $f \leftarrow 0$ | $\triangleright$ Fluss konstant 0 |
| 2: while $\exists$ s-t-Pfad P in (V, A_f) do | $\triangleright$ augmentierender Pfad |
| 3: Erhöhe den Fluss entlang P | $\triangleright$ wie in Beweis zu Satz 3.11 |
| 4: return f | $\triangleright$ maximaler Fluss |



- Laufzeit $O(nmU)$ falls alle Kapazitäten ganzzahlig sind und U die maximale Kapazität ist (ähnliche Idee wie bei Hopcroft-Karp)

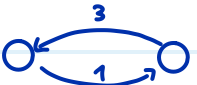
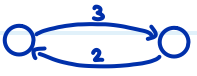
Restnetzwerk: Sei $N = (V, A, c, s, t)$ ein Netzwerk und f ein Fluss.

Dann ist $N_f = (V, A_f, r_f, s, t)$ das Restnetzwerk, wobei

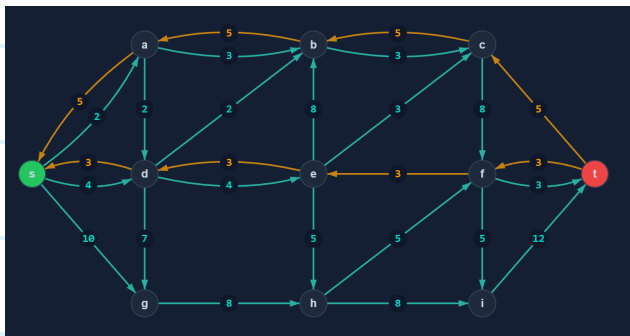
- für jede noch nicht vollständig ausgelastete (d.h. $f(e) < c(e)$) Kante $e \in A$ fügen wir die Kante e zu A_f hinzu mit der noch übrigen Restkapazität $r_f(e) := c(e) - f(e)$
- für jede Kante $e \in A$ auf der etwas fließt (d.h. $f(e) > 0$) fügen wir eine umgekehrte Kante e^{opp} zu A_f hinzu mit dem fließenden Wert $r_f(e^{opp}) := f(e)$
- Wir nehmen an, dass es in N keine einander entgegen gerichtete Kanten gibt

Intuitiv sagt uns das Restnetzwerk, wie wir den aktuellen Fluss verändern können

Aufgabe:

Kante in N	Kante(n) in N_f
	
	
	
	
	

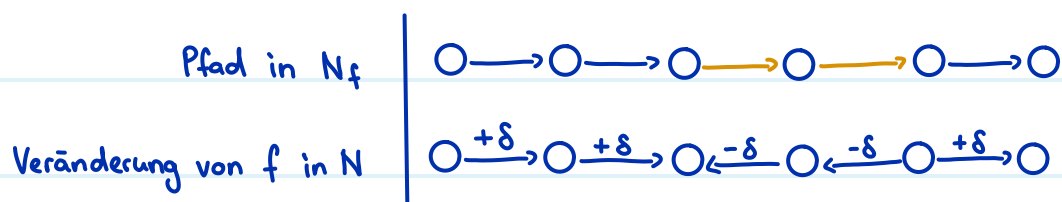
Augmentierende Pfade: gerichtete Pfade von s nach t im Restnetzwerk



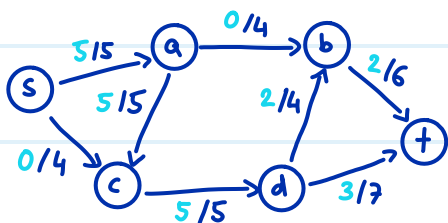
$\delta :=$ minimale Restkapazität einer Kante des augmentierenden Pfades

Für eine Kante e des augmentierenden Pfades P ändern wir f wie folgt.

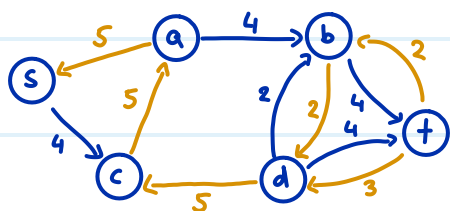
$$f(e) = \begin{cases} f(e) + \delta & \text{falls die Kante in } P \text{ in dieselbe Richtung zeigt wie in } N \\ f(e) - \delta & \text{sonst} \end{cases}$$



Aufgabe: Gegeben sind das folgende Netzwerk und der Fluss:



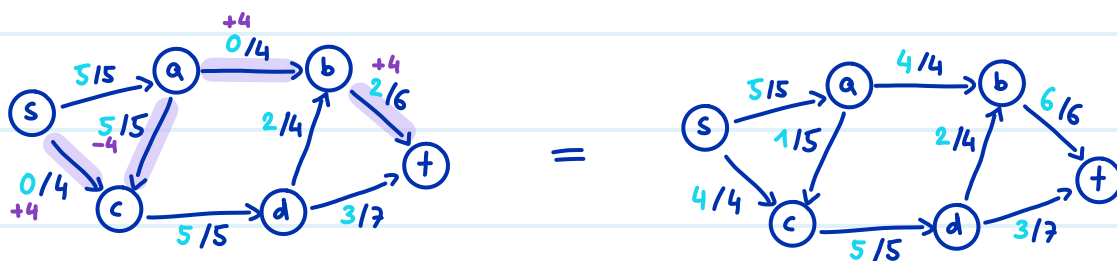
1) Zeichne das Restnetzwerk ein.



2) Finde einen augmentierenden Pfad: (s, c, a, b, t)

3) Bestimme die minimale Restkapazität einer Kante des augmentierenden Pfades: $\delta = 4$

4) Augmentiere den Fluss mit diesem Pfad.



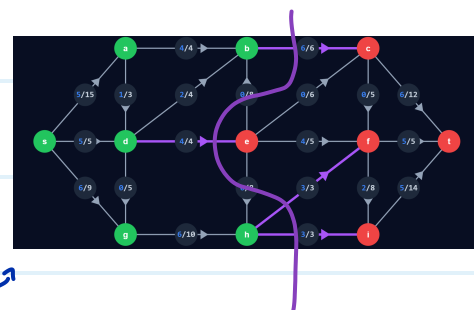
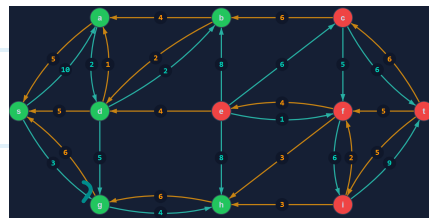
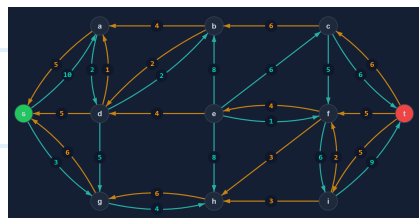
Satz 3.11: f ist maximal $\Leftrightarrow$ es gibt keinen gerichteten s - t -Pfad in N_f

(analog zu Satz von Berge)

Wenn f maximal ist, dann gilt $\text{cap}(S, T) = \text{val}(f)$

für $S = \{v \in V \mid v \text{ ist von } s \text{ aus erreichbar in } N_f\}$

und $T = V \setminus S$



Anwendungen: Verkehr, Strom-/ Wasserversorgung, Internet, Truppenverschiebung, ... (sehr viele)

Für die Prüfung: höchstwahrscheinlich keine Netzwerk-Beweise

höchstwahrscheinlich eine Code Expert Modellierungsaufgabe

Aufgaben.

N Familien nehmen an einem Abendessen teil. Die i -te Familie besteht aus m_i Mitgliedern. Es stehen T Tische zur Verfügung, wobei der j -te Tisch genau s_j viele Stühle hat. Um den sozialen Austausch zu fördern, dürfen keine zwei Mitglieder derselben Familie am selben Tisch sitzen.

Erstelle ein Netzwerk, mit dem sich bestimmen lässt, ob alle Familienmitglieder unter Einhaltung dieser Regel untergebracht werden können.

Sei $N = (V, A, c, s, t)$ ein Netzwerk, wobei

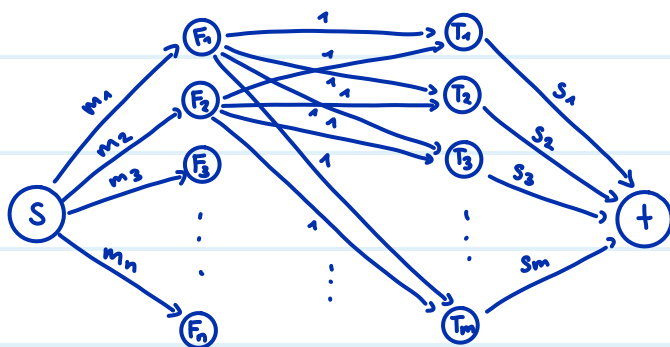
$$V = \{s, t, \underbrace{F_1, F_2, \dots, F_n}_{\text{ein Knoten pro Familie}}, \underbrace{T_1, T_2, \dots, T_m}_{\text{ein Knoten pro Tisch}}\}$$

• A hat folgende Kanten

(s, F_i) mit Kapazität m_i $\forall 1 \leq i \leq n$

(F_i, T_j) mit Kapazität 1 $\forall 1 \leq i \leq n \quad \forall 1 \leq j \leq m$

(T_j, t) mit Kapazität s_j $\forall 1 \leq j \leq m$



$$\text{Zuteilung möglich} \iff \max_{\text{Fluss } f} \text{val}(f) = \underbrace{\sum_{i=1}^n m_i}_{\text{alle Menschen}}$$

In einer Fabrik müssen n Aufträge bis in F Tagen abgeschlossen werden. Jeder Auftrag benötigt genau einen Arbeitstag. Der i -te Auftrag kann jedoch nur zwischen einem Starttag s_i und einem Endtag e_i ausgeführt werden. Ihnen stehen m Maschinen zur Verfügung, und jede Maschine kann höchstens einen Auftrag pro Tag bearbeiten. Erstelle ein Netzwerk, um zu prüfen, ob alle n Aufträge bis zur Frist abgeschlossen werden können.

Sei $N = (V, A, c, s, t)$ ein Netzwerk, wobei

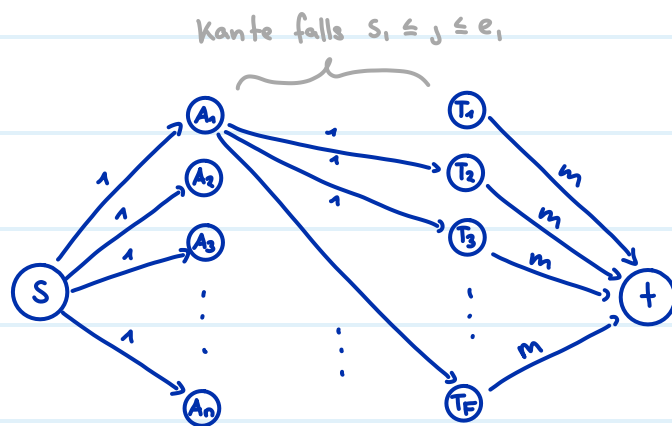
$$V = \{ s, t, \underbrace{A_1, A_2, \dots, A_n}_{\text{ein Knoten pro Auftrag}}, \underbrace{T_1, T_2, \dots, T_F}_{\text{ein Knoten pro Tag}} \}$$

• A hat folgende Kanten

(s, A_i) mit Kapazität 1 $\forall 1 \leq i \leq n$

(A_i, T_j) mit Kapazität 1 $\forall 1 \leq i \leq n \forall 1 \leq j \leq F$ mit $s_i \leq j \leq e_i$

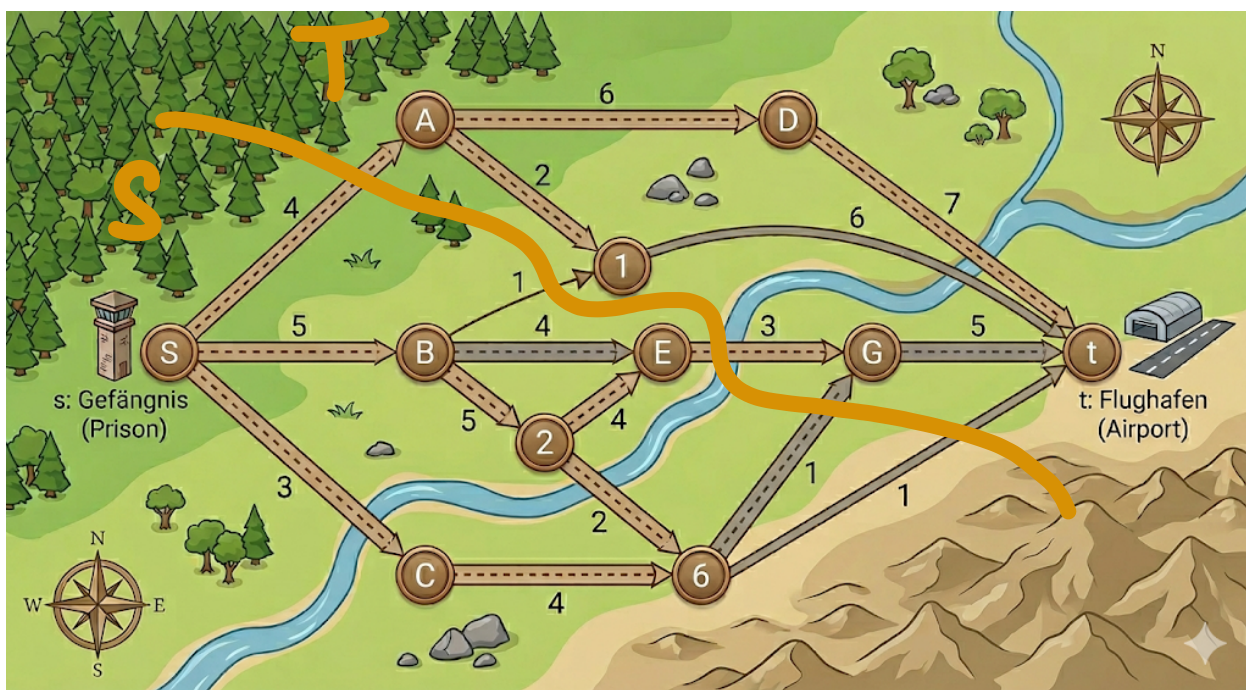
(T_j, t) mit Kapazität m $\forall 1 \leq j \leq F$



Einhaltung der Frist ist möglich $\Leftrightarrow \max_{\text{Fluss}} \text{val}(f) = \underbrace{n}_{\text{alle Aufträge}}$

Ein Dieb bricht aus dem Gefängnis s aus, und möchte zum Flughafen t flüchten. Dabei benutzt er das Strassennetz. Die Polizei verfügt über 10 Polizisten, die sie auf Strassen platzieren können. Für jede Strasse ist angegeben, wie viele Polizisten benötigt werden, um diese zu blockieren.

Ist es möglich, mit 10 Polizisten sicherzustellen, dass der Dieb nicht zum Flughafen gelangt?



Ja es ist möglich, weil $\max_{\text{Fluss } f} \text{val}(f) = 10 \Rightarrow \text{minimaler Schnitt} = 10$